



Airline Reservation & Management System

Object-Oriented Analysis & Design (OOAD)
Complete UML Design Documentation (10 Diagrams)

A semester project submitted for the course
Object-Oriented Analysis and Design (OOAD)

Submitted by:

Laiba Akram (2395), Muskan Abbas (2354), Saira Zaman (2390)

Submitted to: Mam Sadia Sahar

Department: Computer Science

Section: 6th (MB)

Date: 11 June 2026

Table of Contents

1. Introduction (Project Overview).....	3
2. Why We Need This System.....	3
3. Main Users and What They Do.....	3
4. Tools and Technologies Used.....	3
5. System Architecture	4
6. UML Diagrams of the System.....	4
6.1 Use Case Diagram.....	4
6.2 Class Diagram.....	5
6.3 Object Diagram.....	6
6.4 Sequence Diagram.....	7
6.5 Activity Diagram.....	8
6.6 State Machine Diagram.....	9
6.7 Component Diagram.....	10
6.8 Deployment Diagram.....	11
6.9 Package Diagram.....	12
6.10 Network Diagram.....	13
7. How We Used the UML Diagrams in the Advanced Programming Project.....	14
8. Conclusion.....	15

Airline Reservation & Management System

1. Introduction (Project Overview)

The Airline Reservation & Management System is a desktop application built to handle the everyday work of an airline in one single place. Instead of keeping records on paper or in scattered files, the whole process — from a passenger booking a seat to an admin checking the total income — is managed through one program connected to one central database.

The system serves three kinds of users: passengers, employees, and an admin. Everyone logs in through the same login screen and is then taken to their own area of the program, where they can do only the tasks that belong to their role.

This document belongs to our Object-Oriented Analysis and Design (OOAD) course. In OOAD we designed the system by drawing ten UML diagrams. The same design was then implemented as a Windows Forms (.NET) application in our Advanced Programming course. So the two courses are linked: OOAD is where we planned the system with diagrams, and Advanced Programming is where we turned that plan into a working program. This documentation explains each diagram in simple words and, at the end, shows clearly how we used these UML concepts to build the actual application.

2. Why We Need This System

Booking and managing flights by hand is slow and easy to get wrong. Two people can accidentally be sold the same seat, ticket records can be lost, and working out the total earnings means adding everything up manually.

This system solves those problems. It keeps all flights, bookings, and payments in one Oracle database, so the information is always up to date. When a seat is booked, the number of available seats goes down immediately, which prevents double-booking. Reports such as total revenue can be produced automatically as a PDF, which saves time and avoids mistakes.

3. Main Users and What They Do

- **Passenger:** registers an account, searches for flights, books a seat, gives feedback, and views their booked and purchased tickets.
- **Employee:** sells tickets to customers at the counter and completes the tasks assigned to them by the admin.
- **Admin:** adds and manages employees, manages flights, assigns tasks to employees, and views revenue and reports.

One important point: only passengers register themselves. Employee accounts are created by the admin, and the admin account is already set up in the database from the start. This is why the login screen can correctly recognize each kind of user.

4. Tools and Technologies Used

It is useful to separate the tool used to design the system from the technologies used to build it, because the two belong to two different courses.

For the design (OOAD project): all ten UML diagrams were drawn using draw.io (also called diagrams.net), a free online diagramming tool. This single tool was enough to create every diagram in this document.

For building the application (Advanced Programming project), the following technologies were used:

- **Language & framework:** C# on the .NET Framework 4.7.2.
- **User interface:** Windows Forms (WinForms), a desktop graphical interface.
- **Database:** Oracle Express Edition (Oracle XE).
- **Database connection:** ODP.NET (Oracle Data Provider for .NET).
- **PDF reports:** the iTextSharp library, used to create the revenue report as a PDF file.

5. System Architecture

The application is a two-tier client–server desktop program. “Two-tier” simply means there are two main parts: the program running on the user’s computer (the client) and the Oracle database running on a server. There is no website or web server in between — the program talks to the database directly.

When the program starts, the login screen opens one connection to the database and keeps it open. That same single connection is then shared by every screen in the program, which is efficient and keeps the design simple. After a user logs in, the program checks their details against the database and opens the correct area for their role. The role is decided by the login details, not by which computer is used.

6. UML Diagrams of the System

UML (Unified Modeling Language) is a standard set of diagrams used to draw how a software system is built and how it behaves. Each diagram looks at the system from a different angle. Drawing them first helps everyone understand the system clearly before and while the code is written. All ten diagrams below were drawn using draw.io. Each one is explained in simple words, followed by what it shows in our project, how it helped us, and the actual diagram from our project.

6.1 Use Case Diagram

What it is: It shows who uses the system (the “actors”) and what actions they can perform (the “use cases”).

In our system: It shows the three actors — passenger, employee, and admin — with their actions, such as login, search & book flights, sell tickets, manage flights, assign tasks, and view revenue. Login is shared by all three, and “Book Flight” includes “Search Flight”.

How it helped: It gave us a clear, complete list of every feature the program had to provide, so we knew exactly which screens and buttons to build for each user.

Use Case Diagram — Airline Management System

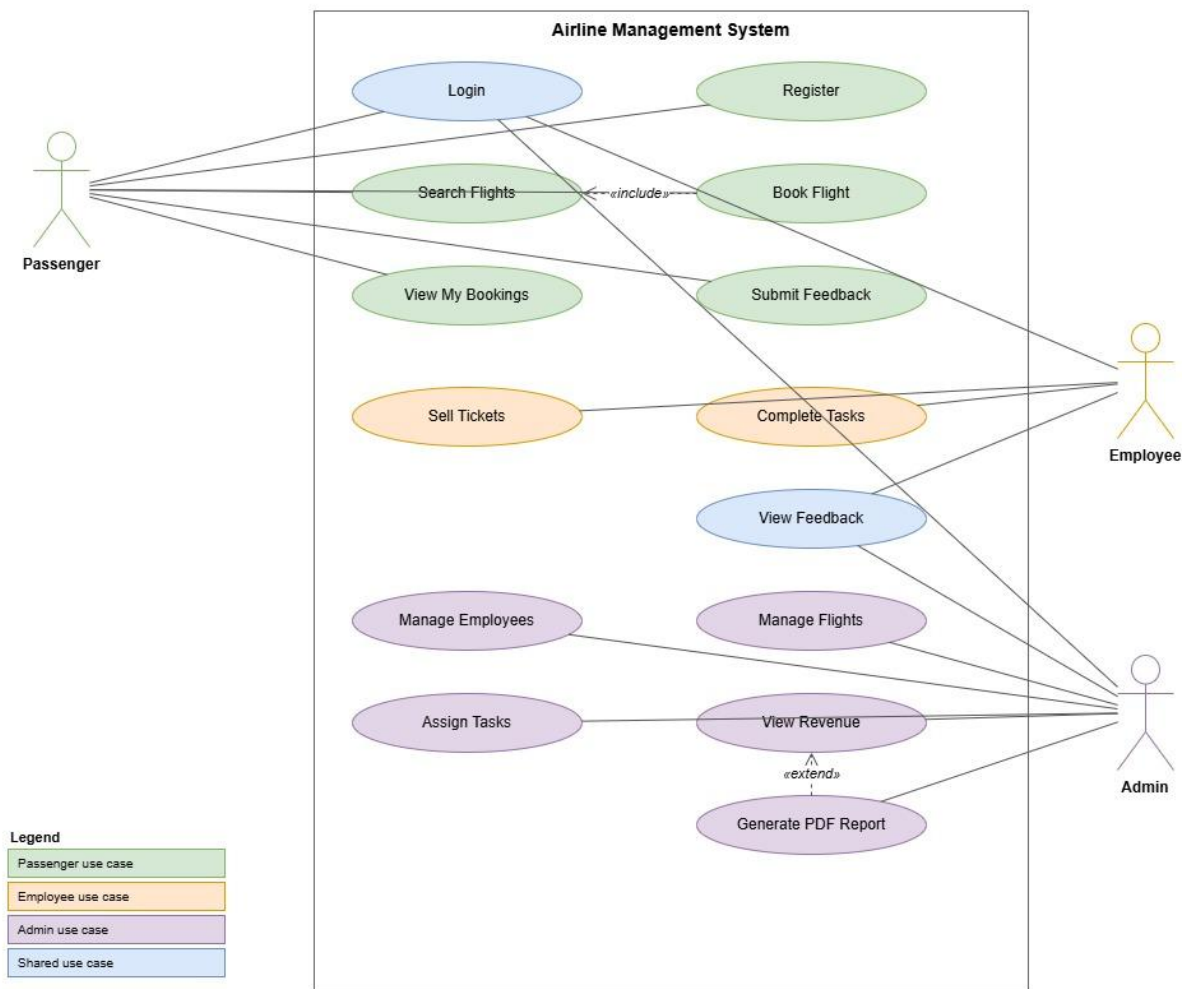


Figure 6.1: Use Case Diagram

6.2 Class Diagram

What it is: It shows the building blocks of the program (the classes), the data they hold, and how they are connected.

In our system: It shows the main entities — User, Flight, Booking, Task, and Feedback — and the screen classes such as SignIn, Passenger_Form, Employee_Form, and Admin_Dashboard, which all share a common Form base. It also shows that a User makes Bookings and that each Booking belongs to a Flight.

How it helped: It became the plan for the actual C# classes and the Oracle tables. Each entity matched a table in the database, and the relationships told us how those tables link together.

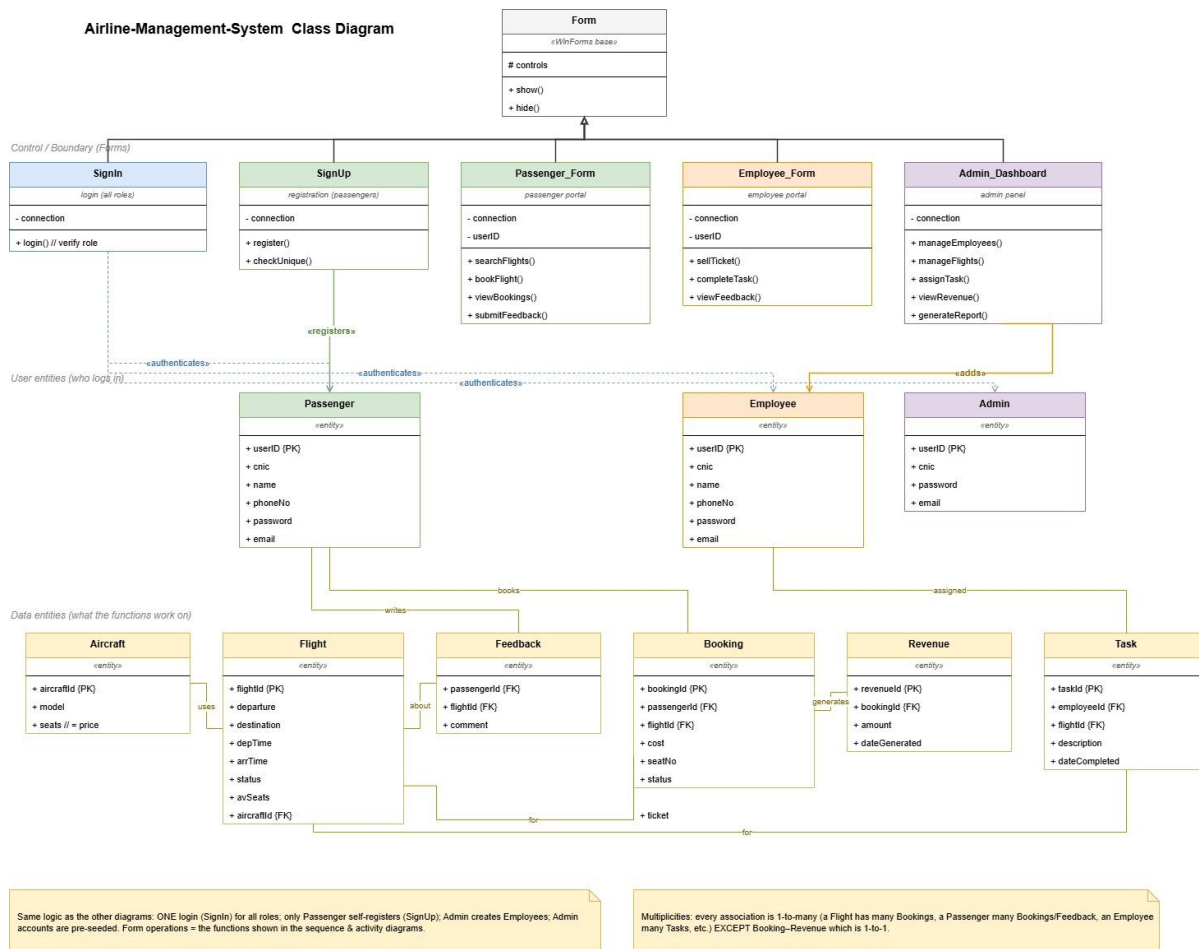


Figure 6.2: Class Diagram

6.3 Object Diagram

What it is: It is a snapshot of the system at one moment, showing real examples (objects) instead of general classes.

In our system: It shows a real example — a passenger who has just booked a seat — together with the matching Booking, Flight, employee, and admin objects, and how they are all linked at that moment.

How it helped: It let us check that the class design really works with actual data, and made it easy to picture how the objects connect while the program is running.

Object Diagram — all three roles (Passenger • Employee • Admin)

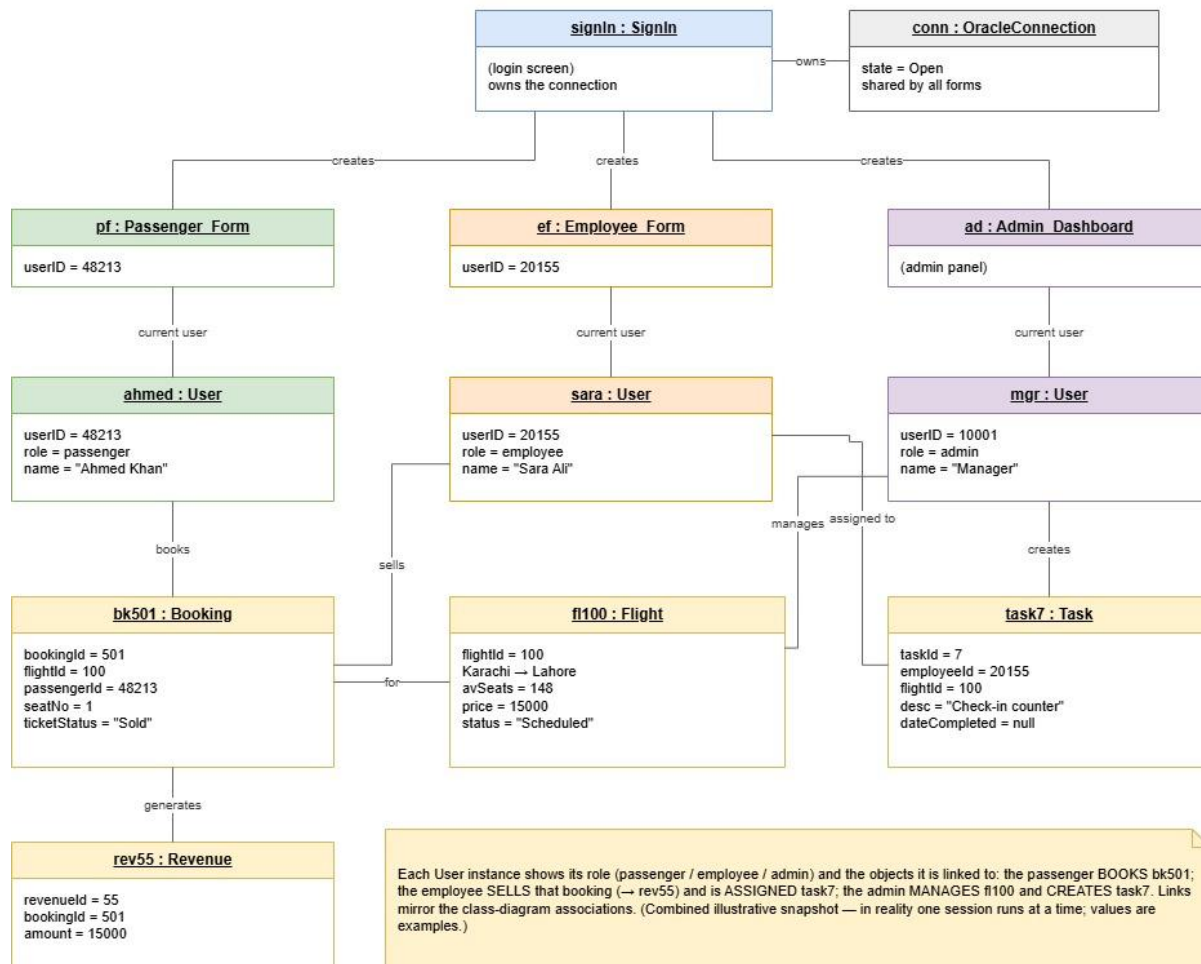


Figure 6.3: Object Diagram

6.4 Sequence Diagram

What it is: It shows the step-by-step messages between the parts of the system, in time order, for one task.

In our system: The login sequence shows the user entering details, the SignIn screen checking them against the database, and the correct portal opening. Other sequences cover booking a seat and selling a ticket.

How it helped: It showed the exact order of steps and which part talks to which, so the methods could be written in the correct sequence.

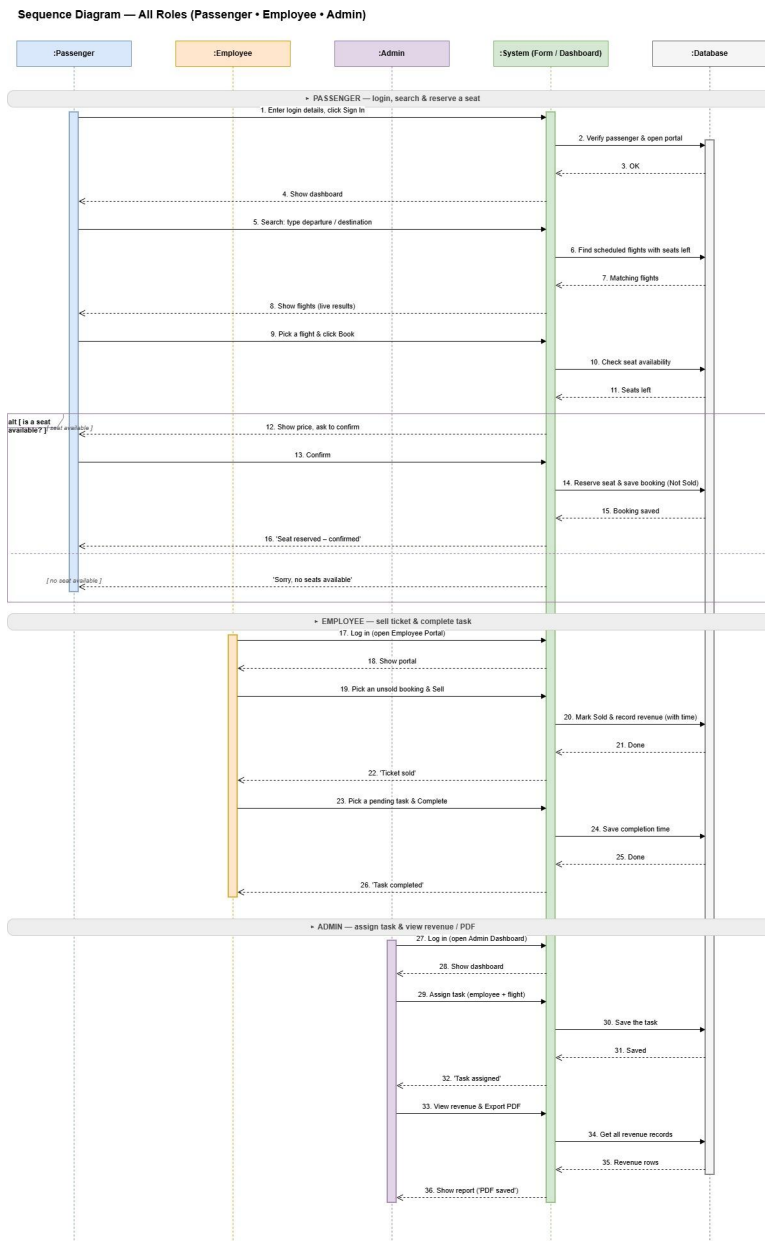


Figure 6.4: Sequence Diagram

6.5 Activity Diagram

What it is: It is like a flowchart that shows the flow of actions and the decisions taken along the way.

In our system: It starts with one shared login and then splits into three lanes for the three roles. For a passenger, it shows the booking flow, including the decision “is a seat available?”.

How it helped: It made the logic and the yes/no decisions clear before coding — especially the booking and seat-checking steps.

Activity Diagram — one shared Login, then different activity per role

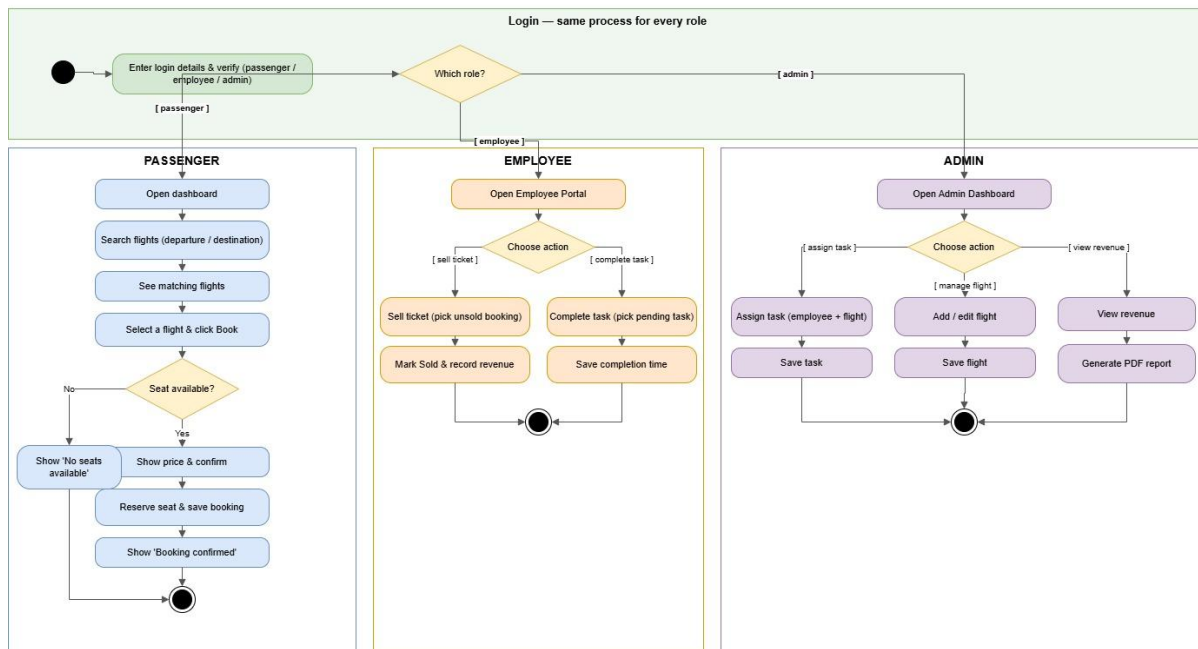


Figure 6.5: Activity Diagram

6.6 State Machine Diagram

What it is: It shows the different states an object can be in and what moves it from one state to another.

In our system: It shows the life of a Booking — from “Unprocessed”, to either “Rejected” (no seat) or “Confirmed”, then “Awaiting Sale”, and finally “Sold” when an employee sells the ticket.

How it helped: It told us exactly which status values a booking and ticket should have in the database, and when to change them.

State Machine Diagram — a Booking (Airline Reservation System)

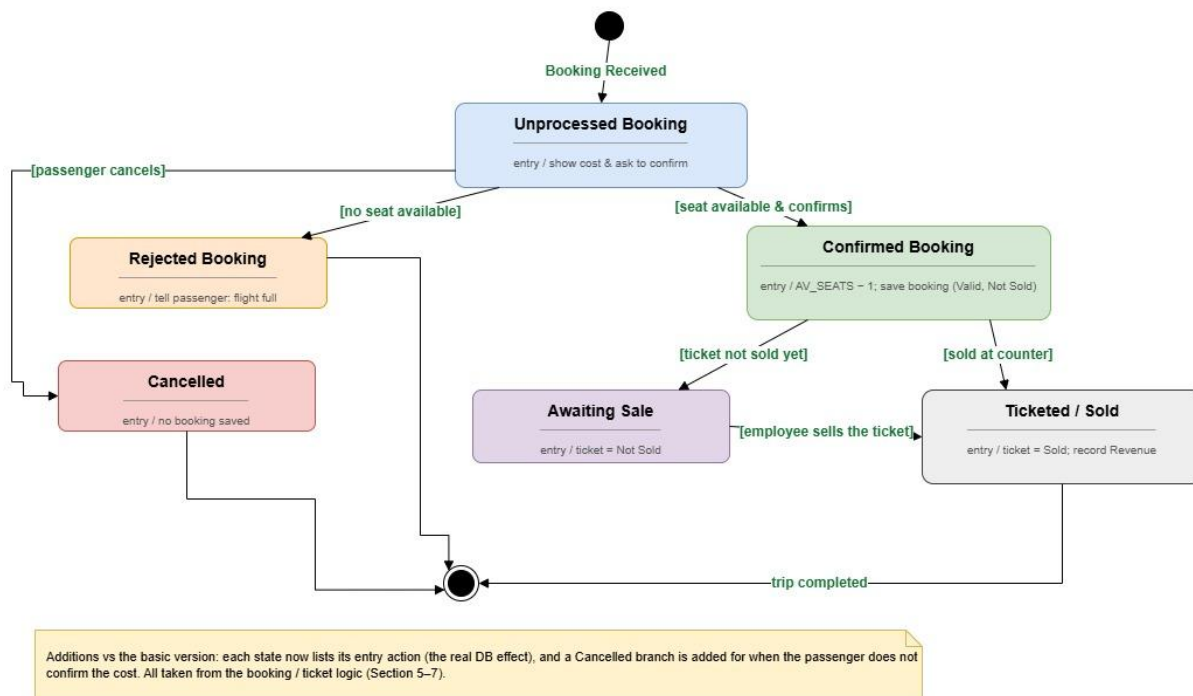


Figure 6.6: State Machine Diagram

6.7 Component Diagram

What it is: It shows the larger software pieces (components) of the system and how they connect.

In our system: It shows the form components, the shared Data Access part (the Oracle connection), the PDF report generator (iTextSharp), and the Oracle database, with the connections between them.

How it helped: It showed how the program is divided into reusable parts, and made it clear that every form uses one shared connection to reach the database.

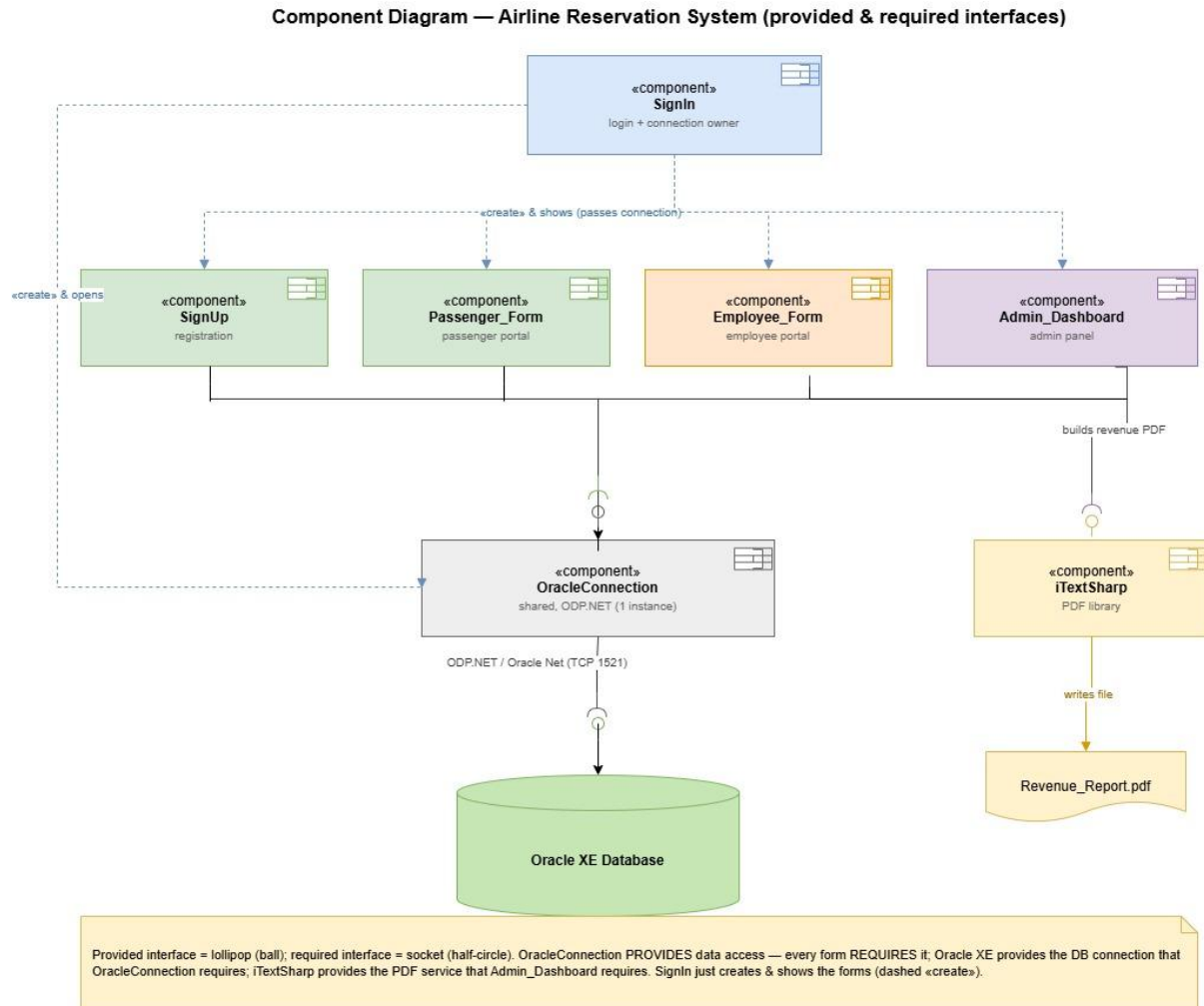


Figure 6.7: Component Diagram

6.8 Deployment Diagram

What it is: It shows the hardware (nodes) and which software runs on each piece of hardware.

In our system: It shows that the whole desktop application — all the forms, the data access, and the PDF generator — runs on one client computer, while the Oracle XE database runs on a separate database server, reached over the network on port 1521.

How it helped: It made clear what needs to be installed where, and that the database lives on a separate machine.

Component Diagram — components deployed on nodes

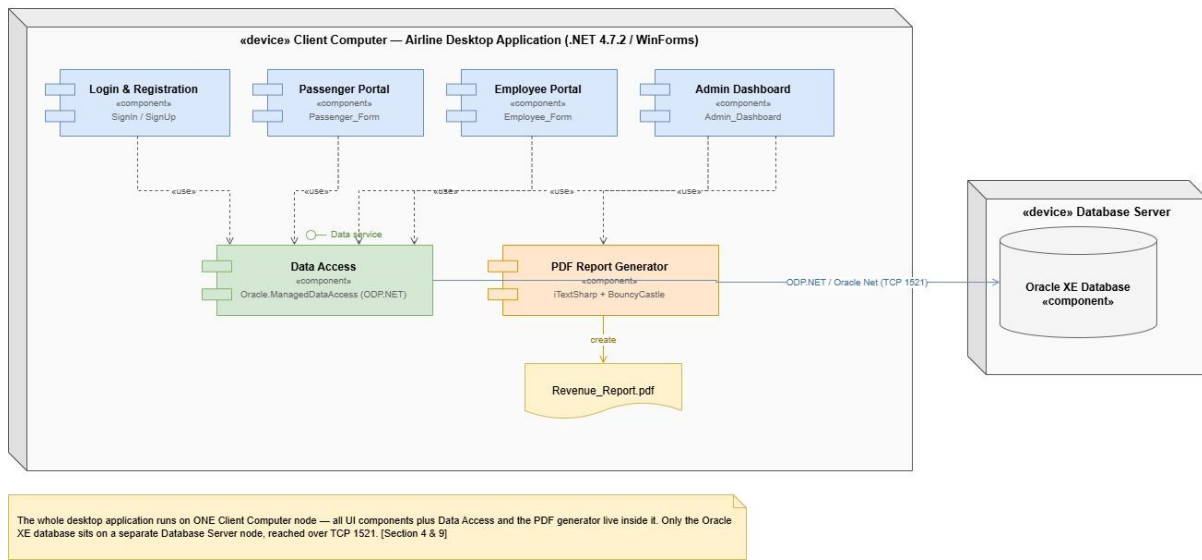


Figure 6.8: Deployment Diagram

6.9 Package Diagram

What it is: It groups related parts of the system into folders (packages) and shows how they depend on each other.

In our system: It groups the parts into Presentation (the forms), Data Access, Domain Entities, Reporting, and the Database. It shows that the forms use Data Access and Reporting, and that Data Access connects to the database.

How it helped: It gave a tidy, high-level map of the whole system and showed the different layers at a glance.

Package Diagram — Airline Reservation System

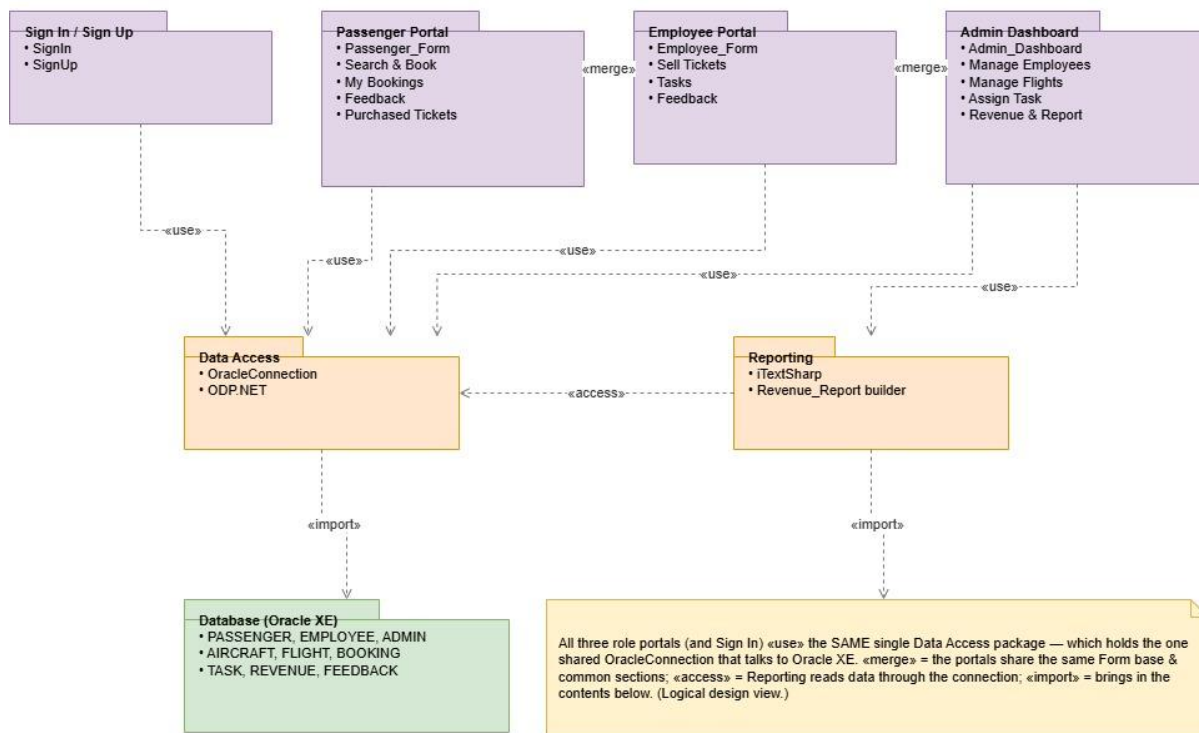


Figure 6.9: Package Diagram

6.10 Network Diagram

What it is: It shows the devices on the network and how they are connected to each other.

In our system: It shows the client computers running the Airline application, connected through a LAN switch to the Oracle XE database server. The connection to the database uses Oracle Net on port 1521.

How it helped: It showed how the client computers reach the central database over the network and where the data actually lives.

Network Diagram — Airline Reservation System (client-server LAN)

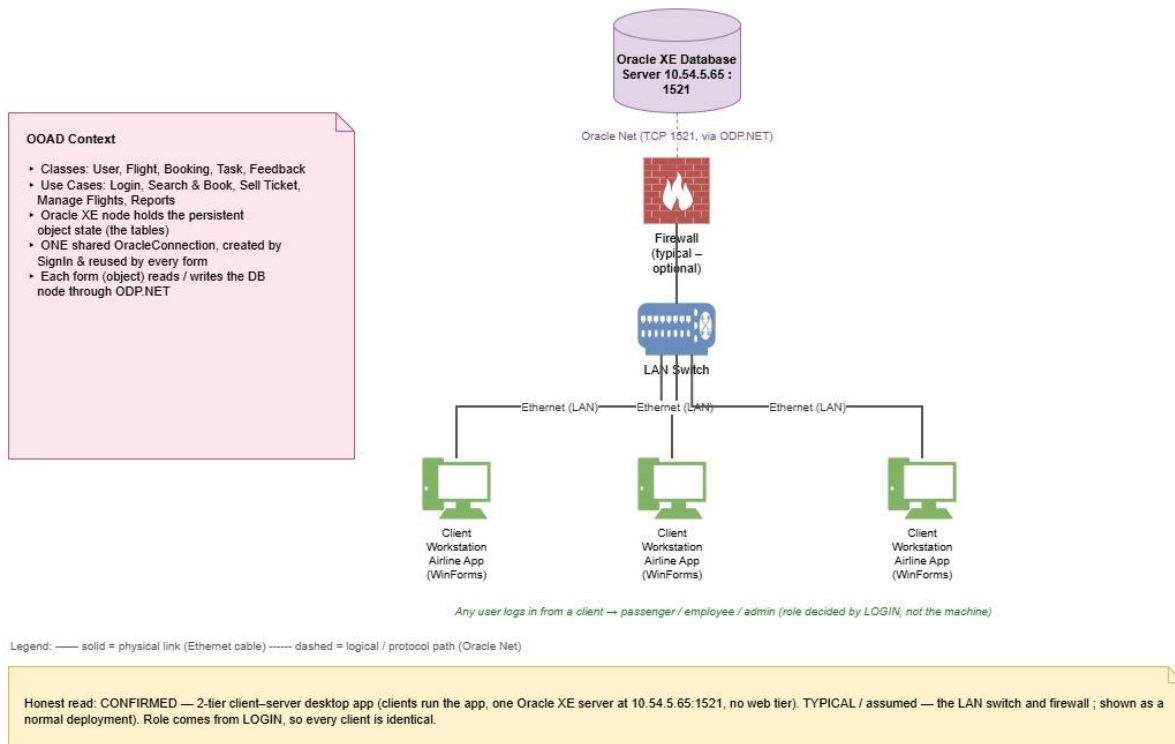


Figure 6.10: Network Diagram

7. How We Used the UML Diagrams in the Advanced Programming Project

In our Advanced Programming course we built the actual application — a Windows Forms program written in C# (.NET 4.7.2) connected to an Oracle database. We did not start from a blank screen. We used the ten UML diagrams from the OOAD design as our ready-made plan, and turned the idea behind each diagram into real code. The points below show, in simple words, how each diagram was used in the application:

- **Use Case Diagram** → the list of actions became the actual buttons and options on each form (for example search & book, sell tickets, manage flights, assign tasks), so each user only sees what their role is allowed to do.
- **Class Diagram** → it became the plan for the C# form classes and the Oracle tables. Each entity (User, Flight, Booking, Task, Feedback) became a table, and the relationships became the links (foreign keys) between those tables.
- **Object Diagram** → it helped us picture a real record — one booking linked to a real user and a real flight — which guided how we save and read rows in the database.
- **Sequence Diagram** → it gave the order of steps for each action, so the code behind a button (such as Login or Book) talks to the database in the correct order.
- **Activity Diagram** → its decisions (for example “is a seat available?”) became the if/else checks in the code.

- **State Machine Diagram** → the booking and ticket states became the status values we store in the database (such as “Not Sold” changing to “Sold”) and told us exactly when to update them.
- **Component Diagram** → it reminded us to create one shared database connection in the login screen and reuse it in every form, instead of opening many separate connections.
- **Deployment Diagram** → it told us to install the application on the client computer, keep Oracle on the server, and use the correct connection string (host and port 1521).
- **Package Diagram** → it helped us keep the code tidy by grouping it into clear parts — the forms, the data access, and the reporting.
- **Network Diagram** → it confirmed how the client application reaches the central database over the local network.

Because the design was ready first, writing the WinForms application was faster and clearer — most coding decisions had already been answered by a diagram. This is exactly how UML is meant to be used: design the system in OOAD, then build it in code. In this way our OOAD work directly supported and shaped our Advanced Programming project.

For completeness, the actual steps we carried out to build the application were: (1) designing all ten UML diagrams in draw.io; (2) creating the Oracle XE database tables from the class diagram; (3) building the Windows Forms screens — SignIn, SignUp, and the passenger, employee, and admin portals — in C#; (4) writing the logic behind each button by following the sequence and activity diagrams; (5) opening one shared database connection in the login screen and reusing it across every form; (6) saving and updating the booking and ticket status values exactly as shown in the state diagram; and (7) adding the iTextSharp library to generate the revenue report as a PDF. Listing these steps keeps the design-to-code journey clear and complete.

8. Conclusion

The Airline Reservation & Management System brings the entire ticketing process into one easy-to-use desktop program backed by a central Oracle database. The ten UML diagrams let us understand and plan the system from every angle — who uses it, what it contains, how it behaves, and how it is deployed — both before and during coding. Together, they turned the idea into a clear design and then into a complete, working project for the Object-Oriented Analysis and Design (OOAD) course.